

Lappeenrannan teknillinen yliopisto
School of Business and Management

Software Development Skills

Diako Jalal, 2524490

LEARNING DIARY, FRONT-END MODULE

LEARNING DIARY

22.6.2026

Sessions 1 and 2 - Setting up the tools, and HTML that means something

LUT lectures 1 and 2, HTML parts I and II, and a practice page built from a blank file with nothing open to copy from: heading hierarchy, paragraphs, a list, a link, an image with alt text, and a form with text, number and date inputs.

I learned about,

- semantic HTML. I had been reaching for a div for everything. Using header, nav, section and footer costs nothing, and the file is legible when I come back to it two days later.
- the viewport meta tag. It tells the browser to lay the page out at the device's own width instead of pretending to be a 980px desktop and zooming out. Without it, media queries do nothing at all on a real phone, and nothing anywhere warns you about it.
- labels. The for attribute has to match the input's id exactly, or clicking the label does nothing.
- filenames. Lowercase and no spaces, because Windows does not care about case and a Linux server does, and I would rather not find that out later.

The first thing that went wrong was small and stupid. None of my labels did anything when clicked, because I had matched the for attribute to each input's name rather than its id. They look interchangeable in the source and they are not.

Tags I had to look up: fieldset, and the input types for number and date. I also deliberately deleted a closing div on one section to see what happens, and the rest of the page climbed inside it, which is worth having seen once on purpose rather than at midnight.

25.6.2026

Sessions 3 and 4 - CSS basics, the box model and Flexbox

LUT lectures 4 & 5, CSS 1 & 2. Styled just like the previous session and then intentionally broke it so the box model acted up. Then a navigation built entirely with Flexbox and finished at the same time as Flexbox Froggy.

I learned about,

- the box model. An element has its content inside it, then its padding inside that, then its border. Outside all of this is a margin. When setting the width, it normally just defines the content width by default, so 100% width with 20px of padding makes it larger than its containing element so it overflows.
- box-sizing: border-box. It modifies what the width value actually specifies, so that padding and border are included within it instead of added outward. One line and the overflow disappeared too, saving me a significant proportion of the layouts I once feared would haunt me.
- specificity. A rule with .card p beats a rule with p, and an inline style beats both.
- margin collapse. Two stacked elements with 20px margins will result in 20px of space, not 40. I spent about ten minutes thinking this was a bug.
- Flexbox is one dimensional. Justify-content affects how items are distributed along the main axis, and align-items affects them across it, and then using flex-direction:column flips the axes.

Finally, to make matters worse, my stylesheet didn't do anything for the first 10 minutes. The file was style.css and I had linked styles.css. DevTools, Network tab, 1 404, which I should have opened before doing anything else.

After a while I kept editing CSS and seeing nothing, which was the browser serving the file from cache.

Ctrl+Shift+R. I fell for that three more times before it hit.

One rule I couldn't get to work was when a colour on p .content p was quietly winning, there was inspection showing the losing rule, with a strike through which I learned about rather than grabbing for !important. Furthermore justify-content and align-items were the pair I kept

confusing and solved problems in my understanding by thinking about the main axis and cross axis instead of horizontal and vertical.

28.6.2026

Sessions 5 and 6 - CSS Grid, and tutorial parts 1 and 2

Built a three-column card grid, then Grid Garden. After that I started the Traversy "Professional Website From Scratch" video, parts 1 and 2: environment setup, base HTML and the core CSS. I typed all of it rather than pasting, which is slower and is the point.

I learned about,

- what 1fr means. It is one share of the space left over, not a fixed unit, so `repeat(3, 1fr)` gives three equal columns at any container width.
- the difference between a grid gap and a margin. gap is the grid's own spacing between tracks and only ever appears between them, so there is no stray space at the outer edges and no margin collapse to reason about. A margin belongs to the item and applies whether or not there is a neighbour.
- `repeat(auto-fill, minmax(280px, 1fr))`, which reflows on its own without a single media query. Probably the most useful line I have learned so far.
- CSS custom properties. A variable declared in `:root` and used as `var(--primary)`. Putting colours in variables means one edit changes the whole site instead of a find-and-replace across the file, and the name records what the colour is for rather than what it looks like.

My variables silently did nothing for a while. I had declared them inside the `.btn` rule rather than in `:root`, so they only existed within that selector and every other use fell back to nothing.

There is no error and no warning: the property simply does not apply, which makes it look like a typo somewhere else. Moving the block to :root fixed it.

Grid turned out to be easier than Flexbox, which I did not expect. Handling two dimensions at once is somehow less confusing than one dimension that changes meaning.

2.7.2026

Sessions 7 and 8 - Tutorial parts 3 and 4, buttons, utility classes and cards

Parts 3 and 4 of the tutorial. The pure-CSS hamburger icon and the button styles, then the card section laid out with Grid, which was a second pass over week one's material inside a real layout.

I learned about,

- pseudo-elements. ::before and ::after generate an extra box inside an element that does not exist in the HTML. The hamburger is one element plus its two pseudo-elements making three bars, and a class rotates two of them into an X. I could use one for the little open and close arrow on the FAQ questions later, and for the required-field asterisks on the form.

- utility classes. One .btn reused everywhere rather than .hero-button, .footer-button and .form-button all repeating the same declarations. The stylesheet got shorter while the page got bigger, which I had not expected.

::before refused to appear at all. It needs content: "" to exist, and an empty string still counts as content; without the property the pseudo-element is never generated. There is no error, the bar is simply absent, so I spent twenty minutes checking the rotation maths instead of the one line that mattered.

I also narrowed the browser on the finished card grid and watched three columns crush into unreadable slivers rather than wrapping, because the column count is fixed at three regardless of width. I am not fixing it yet, that is the media query session, but having seen the failure first should make the cure make more sense.

6.7.2026

Sessions 9 and 10 - Tutorial parts 5 and 6, accordion, mobile menu and responsiveness

Part 5, the FAQ accordion and the footer, which is the first real JavaScript in the build. Then part 6, the media queries and the slide-in mobile menu.

I learned about,

- why you cannot transition to height: auto. The browser has to interpolate between two numbers, and auto is not a number, it is resolved at layout time, so there is nothing to animate towards. The tutorial animates max-height instead.
- the trade-off with the max-height trick. You have to guess a value larger than the content will ever be, and the animation then spends part of its duration travelling through empty space, so opening feels slower than closing. Guess too small and the content is clipped with no warning. It is a workaround, not the right answer.
- media queries, and that they belong at the bottom of the stylesheet. At equal specificity the later rule wins, so a media query placed above the rule it is meant to override does nothing at all.
- picking one direction and staying in it. I used max-width throughout, desktop first, like the tutorial, rather than mixing max-width and min-width in one file.

The breakpoints are 768px and 500px. Not arithmetic: 768 is where the three-column grid stops being readable and 500 is where the nav links stop fitting, so I took the numbers from where the layout actually broke rather than from a list of devices.

The fixed mobile menu positioned itself inside the header instead of the viewport, which took the longest. This one is genuinely obscure: backdrop-filter on the header creates a containing block, so position: fixed children stop being positioned relative to the viewport. The same is true of transform, filter and perspective. I found it by deleting properties on the parent one at a time until the menu jumped where it should be. What broke first when I narrowed the window was the nav, well before anything else.

10.7.2026

Sessions 11 and 12 - The deployment video, and JavaScript fundamentals

Watched part 7, the deployment section, and took notes rather than following along, because I had no repository to deploy to and did not want to stop the build to sort that out. Then LUT lecture 3, JavaScript part I, practised in the browser console before touching a file.

I learned about,

- relative and absolute paths, which the video makes a point of. A leading slash means the root of the domain, which on GitHub Pages is username.github.io and not the repository folder, so /css/style.css would point at a file that does not exist. Written as css/style.css it works in both places. Locally the root and the project folder happen to be the same directory, so the mistake is invisible until the site is live, which is why I wrote it down.

- type coercion. "2" + 2 is "22" but "2" * 2 is 4, because + is also string concatenation and * is not. Input fields always hand you strings, so parseFloat before any arithmetic. This will matter for the hours field.

- = assigns, === compares, and == compares after converting types. I am going to carry on as though == does not exist.

- const not meaning immutable. `const a = []`; `a.push(1)` is perfectly legal, because const only stops the variable being reassigned, not the contents being changed.

My first attempt at totalling hours returned "0234" instead of 9. The test values had come out of an array as strings, so + concatenated them one after another rather than adding them, and the loop ran perfectly and produced nonsense. Nothing throws, which is the unnerving part. `parseFloat` on the way in fixed it. The console is genuinely the fastest feedback loop I have had so far, and I should have started there rather than in a file.

Watching a deployment video without deploying anything is exactly the trap the course warns about, and I am aware of it. The build is going well and I would rather keep momentum than spend an evening on version control, so the repository is a job for the end.

14.7.2026

Sessions 13 and 14 - The DOM, and arrays as data

`getElementById`, `textContent`, `innerHTML` and `addEventListener`. A button that changes text on the page, then a form that reads its inputs and displays them. After that, `push`, `filter` and `sort` on an array of objects shaped like a `StudyLog` entry, plus a function that totals the hours and one that returns only the entries for a given subject.

I learned about,

- why script placement matters. A script in the head runs before the HTML below it exists, so `getElementById` returns null and the next line throws "Cannot read properties of null". Moving the tag to the bottom of the body, or adding `defer`, fixes it. I hit this within ten minutes of starting.

- `preventDefault()`. A submit button's default behaviour is to send the form off to a URL and reload the page, which throws away everything held in memory. `preventDefault()` stops that built-in behaviour from running so my own handler can deal with the data instead. It prevents the browser's default action for that event, not my code.

- single source of truth. The entries array is the data and the page is only a picture of it. Something changes the array, then the list is redrawn from the array. I never read a value back out of the page to work out what the data is, because then there are two versions of the truth and sooner or later they disagree.

- filter, which was the most useful method by far. The subject filter and deleting an entry turn out to be the same operation: build a new array without the things I do not want.

Beyond the null error, my form handler ran once and then the page reloaded and everything vanished, which was the missing `preventDefault`. I also spent a while logging an array that was always empty, because I was pushing inside a function I had defined and never called.

18.7.2026

Sessions 15 and 16 - Web storage, and dates

LUT lecture 15, Web storage, which is buried at number 15 in the playlist and which I nearly skipped. Then the date handling StudyLog needs: today's date, the last seven days, and a readable date on every entry.

I learned about,

- why `localStorage` needs JSON. It stores strings and nothing else. Save an array straight in, read it back and you get "[object Object]", because the browser quietly calls

toString on it. JSON.stringify going in and JSON.parse coming out, every time, no exceptions.

- JSON.parse(null) returning null rather than throwing, so the very first load needs a check before use rather than a try/catch around it.

- the limits. Roughly 5MB, synchronous, strings only, and scoped per browser and per origin, so Chrome cannot see what Firefox saved and nor can another machine. To sync between two computers I would need a server with a database behind it and some form of account, which is a different course entirely.

- getMonth() being zero-based while getDate() is not, which is not a rule so much as an accident that has annoyed people for thirty years.

The timezone bug, which was the best thing I did all month. new Date().toISOString().slice(0,10) looks like the obvious way to get today's date as "2026-07-18". toISOString converts to UTC, and Finland is UTC+3 in summer, so between midnight and 03:00 local time it hands back yesterday. The symptom was an entry added late at night filing itself under the previous day and the streak breaking for no visible reason. I found it by setting my computer's clock to 00:30 and reloading, at which point it happened on demand instead of at random. The fix is to build the string from getFullYear, getMonth() + 1 and getDate() with padStart, all local. Next time the tell is any function with UTC or ISO in its name being used for something that is supposed to be local.

Saving also mysteriously failed at first, until I noticed the address bar said file:// because I had opened index.html directly instead of through Live Server.

22.7.2026

Sessions 17, 18 and 19 - Planning, the skeleton, and making it work

Planned StudyLog on paper before writing anything: the sections, the feature list, and what a single entry needs to store. Then the whole HTML structure with real content and no JavaScript, then the CSS shell at desktop width only, then the JavaScript one step at a time with each step tested before the next.

I learned about,

- cutting scope. I cut editing an existing entry, export to CSV and a monthly calendar view. Add, delete, list and total is the core, and everything else is decoration on top of something that has to work first. Cutting hurt less than I expected once I had written the list down and could see how much of it was optional.
- what one entry has to store: id, subject, date, hours and notes. The id is `Date.now()` and exists purely so that delete has something reliable to match on.
- building in testable steps. Log the form values, then push into the array and log the array, then draw the list, then delete, then save and load, then the totals, then the chart, then goals, then the filter. Nine steps, each one working before the next was started, which meant every bug had at most one possible cause.
- where Grid goes and where Flexbox goes. Grid for the page sections, because I am placing things in two dimensions and want the tracks to line up across the whole section; Flexbox for the rows inside them, because that is one line of items that needs spacing and alignment and nothing more.

Delete removed the wrong row as soon as a filter was active, because I was passing the array index and the filtered list's indexes no longer matched the real array's. Matching on the entry's id fixed it permanently. Then everything vanished on refresh, because I had written the save function and never called it, so it now runs after every single change. Then delete stopped responding after a redraw, because rebuilding the list with `innerHTML` throws the old elements away and their event listeners with them.

The chart took longest. The bars were invisible, and it turned out their heights are percentages while the container had no fixed height, and a percentage needs something to be a percentage of. Giving the container a height in pixels fixed all of them at once.

27.7.2026

Sessions 20 and 21 - Responsive layout, testing, and shipping

Added the media queries so the sections collapse to one column and the nav becomes a menu button, then tested deliberately trying to find problems rather than trying to see it work. After that, the part of the course I had been putting off since 10.7: set Git up properly, made the GitHub account and a public repository, put the whole project into it, turned on Pages, wrote the README, recorded the demo video with OBS and submitted the link.

I learned about,

- testing as an activity separate from building. The device toolbar at 375, 768 and 1440px, then submitting the form empty, then with letters in the hours field, then with a date in the future, then adding ten entries and deleting all of them and reloading, then opening it in a browser I had never used for it.
- validation being about what a user can type, not what I would type. Empty and letters I had thought about; a date in the future I had not, and it happily recorded four hours of study for next March.
- the four places a file lives in Git: the working directory, then the staging area after git add, then local history after git commit, then GitHub after git push. I had assumed committing put it online. It does not, and understanding that there are four states rather than one is what made the error messages readable instead of arbitrary.
- the debugging order actually being a method. Read the error properly, then log the thing I am assuming is fine, then comment out half the code. Almost everything died

at step two, and nine times in ten the value was undefined, an empty array, or a string where I expected a number.

The hours field accepted 0 and negative numbers, so a session could subtract from my total. Fixed with min="0.5" on the input and a check in the JavaScript as well, because the HTML attribute alone is trivially bypassed. The empty state was also blank rather than saying anything, which reads as broken instead of empty.

Git refused to commit at all until I set `git config --global user.name` and `user.email`, and then the push was rejected: it asked for a password and then would not take my actual GitHub password, because GitHub removed password authentication in 2021. I generated a Personal Access Token and used that as the password. The whole project then went up as a single commit, so my history is one entry called "initial commit" rather than a record of the build, which is a fair description of what I did but not a useful one.

Pages then failed four times in a row. A 404 for the first two minutes, which was only the build running. Then the page loaded with no CSS at all, which was the absolute path I had written down on the tenth and still got wrong three weeks later. Then it worked on my machine and the logo 404'd live, because the file is `Study.svg` and I had linked `study.svg`: Windows does not care about filename case and the Linux server does, and I only found it by opening the Network tab on the live site and reading the request that failed. Then a push that appeared to change nothing, which was the cache again, confirmed by checking the Actions tab to see whether the deploy had actually run. Finally I opened the link in a private window, which is the only honest test of whether the repository is really public.

Leaving the repository until the last day was the clearest mistake of the whole month. Everything above is ordinary and every one of these problems has a two-line answer, but they all arrived on the same evening, on top of the testing, with nothing left to absorb them. Version control also would have let me undo the afternoon where I broke the chart, instead of retyping it from memory.

Looking back over the four weeks, at the start I could read HTML and CSS and assumed the gap between that and building a site was mostly memorising more properties. It is not. It is knowing where the data lives, what runs in what order, and how to find out why something is not doing what I told it. If I started again I would set the repository up on day one and commit at the end of every session, type more and watch less, and start the project earlier: the fortnight from session 17 taught me more than the fortnight before it, because a blank file asks questions a video never does. Next I would add editing an existing entry, and replace localStorage with a real backend so the data follows me between machines.